

Reviewer Pack — Minimal Rank of Geometric Invariants in Representation Theor...

Entry 8f01344baf53 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 15:19:47 UTC

Minimal Rank of Geometric Invariants in Representation Theory vs Monotone Circuit Lower Bounds

Entry ID: 8f01344baf53 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 15:19:47 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory: Geometric Invariant Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For a given representation V of a finite group G , let $R(V)$ denote the rank of the vector space V . For every instance of the k-CLIQUE problem with n vertices, there exists a set $S \subseteq V$ such that $R(S) \geq \Omega(n^{2/k})$, ‘where $R(S)$ is the rank of the subspace spanned by S .’]

Rationale (proposer’s reasoning):

[‘Geometric Invariant Theory provides tools to study the geometric properties of representations. By analyzing the rank of subspaces in a representation, we may uncover intrinsic complexities that are not revealed by traditional circuit complexity measures.’, ‘If this conjecture holds, it would suggest a new way to understand the hardness of k-CLIQUE using representation theory, potentially leading to novel approaches for proving monotone circuit lower bounds.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 125386e4bf806253

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the random graphs G with n vertices and their adjacency matrices A , the minimum rank $R(S)$ across all subspaces S of dimension at least d meets or exceeds $\Omega(n^{2/k})$. The conjecture is falsified if this condition is not met for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    k = random.randint(2, n-1)
    d = random.randint(1, n//2)

    # Generate a random adjacency matrix for a graph with n vertices
    A = [[random.choice([0, 1]) if i != j else 0 for j in range(n)] for i in range(n)]

    # Compute the rank of the subspace spanned by all subspaces of dimension at least d
    min_rank = float('inf')
    for i in range(2**n):
        S = [j for j in range(n) if (i >> j) & 1]
        if len(S) >= d:
            # Compute the rank of the subspace spanned by S
            submatrix = [[A[i][j] for j in S] for i in S]
            rank = gaussian_elimination(submatrix)
            min_rank = min(min_rank, rank)

    metric_value = min_rank
    instances_tested = 1

    # Check if the conjecture holds
    conjecture_holds = min_rank >= (n**2 / k) * math.log(2, n)
    counterexample = "min_rank too small" if not conjecture_holds else ""

    return {
        "metric_name": "Minimum Rank",
        "metric_value": metric_value,
```

```

        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0

    for j in range(n):
        i_max = -1
        for i in range(rank, m):
            if matrix[i][j] != 0:
                i_max = i
                break

        if i_max == -1:
            continue

        matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]

        for i in range(m):
            if i != rank and matrix[i][j] != 0:
                factor = matrix[i][j] / matrix[rank][j]
                for k in range(n):
                    matrix[i][k] -= factor * matrix[rank][k]

        rank += 1

    return rank

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']}\n first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18498
2	propose	ollama_remote	glm4:latest	0	0	13511
3	preregistration	ollama_remote	glm4:latest	0	0	9401
4	novelty	ollama_remote	glm4:latest	0	0	8861
5	novelty	ollama_remote	glm4:latest	0	0	8676
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12280
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10708
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11078
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11027
10	judge	ollama_remote	glm4:latest	0	0	11458

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115497 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/8f01344baf53.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/8f01344baf53.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/8f01344baf53.tar.gz](#) (if generated)